

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen

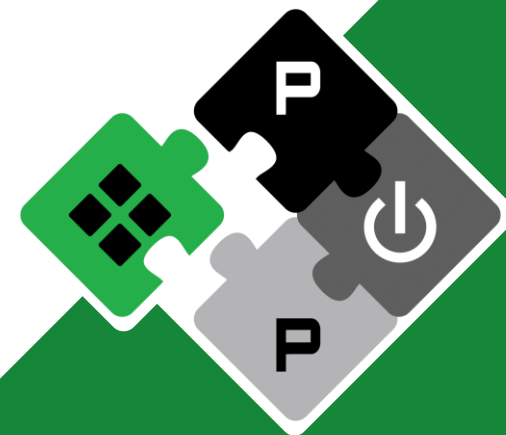
zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli
Luca Benini

avanelli@iis.ee.ethz.ch
lbenini@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!



@pulp_platform



pulp-platform.org



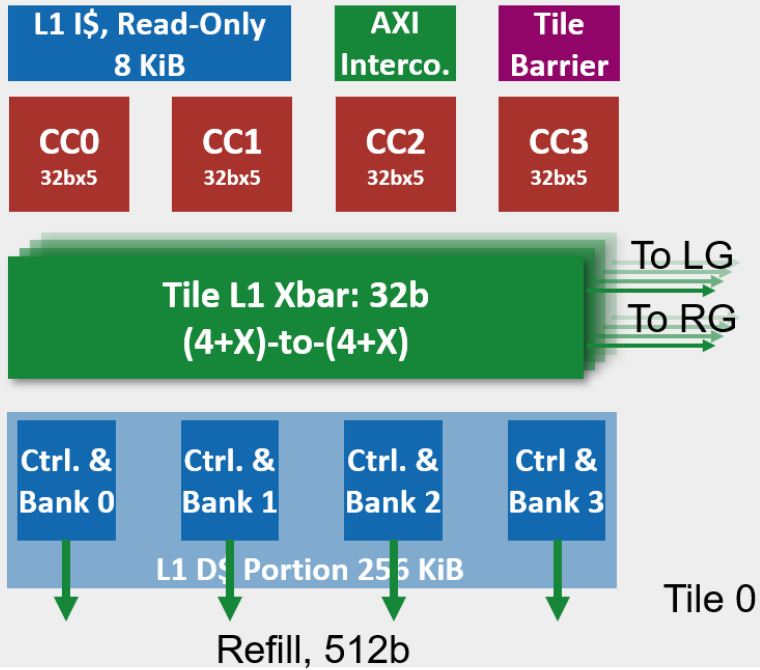
youtube.com/pulp_platform



Hardware Development (Cluster)



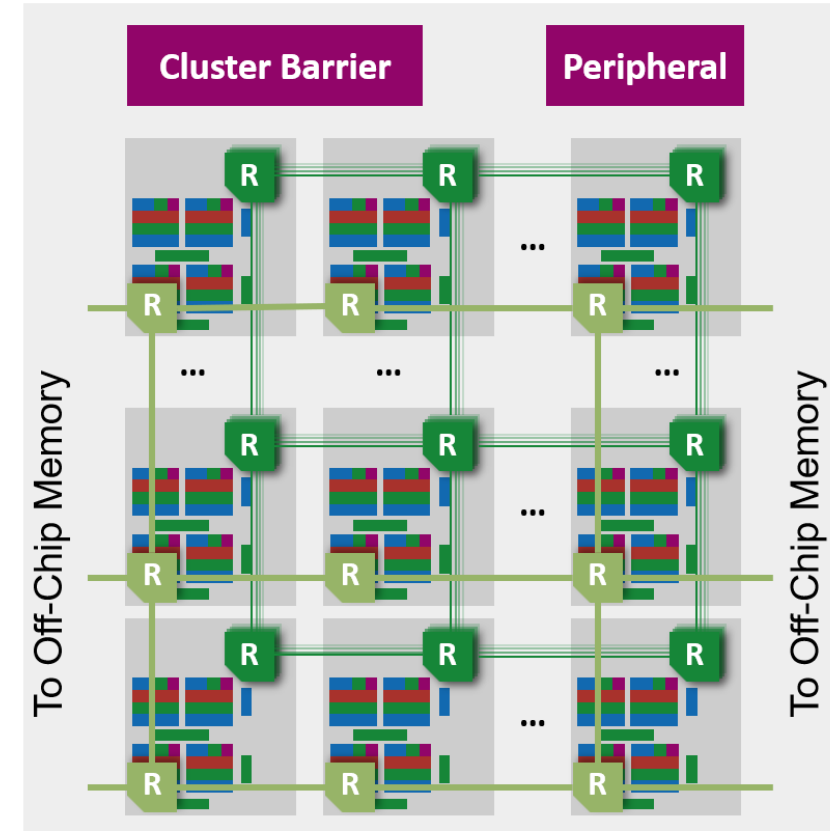
Tile



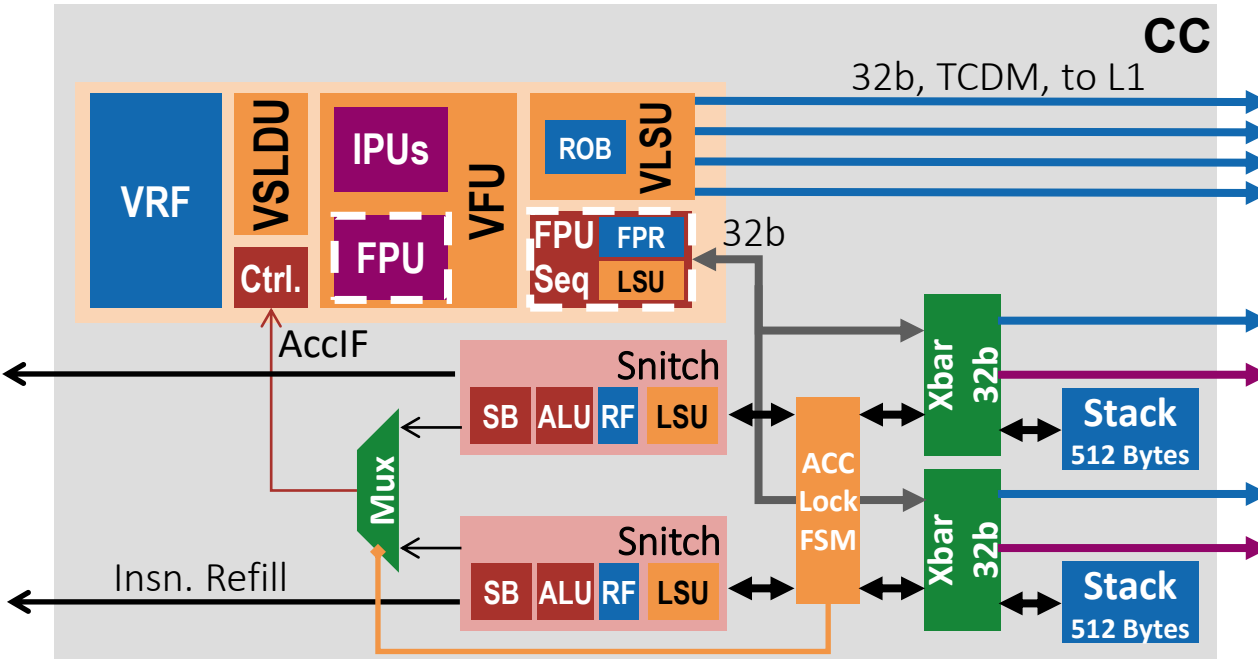
Group



Cluster



Hardware Development (Cluster)

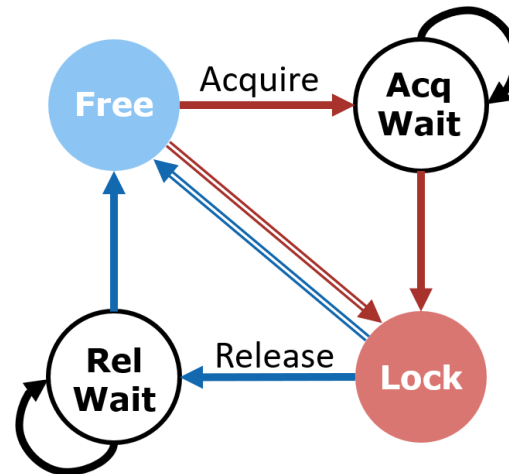


• MISCs and Bug Fix

- Fix several bugs when enabling FPU configurations
- Add dual-scalar configuration into CI

• WIP: Implement multi-barrier support for dual-scalar core

- This is needed because now we have different scalar cores working on different tasks.
- Based on existing partial barrier logic, using write data to set multiple masks.
- Under testing now



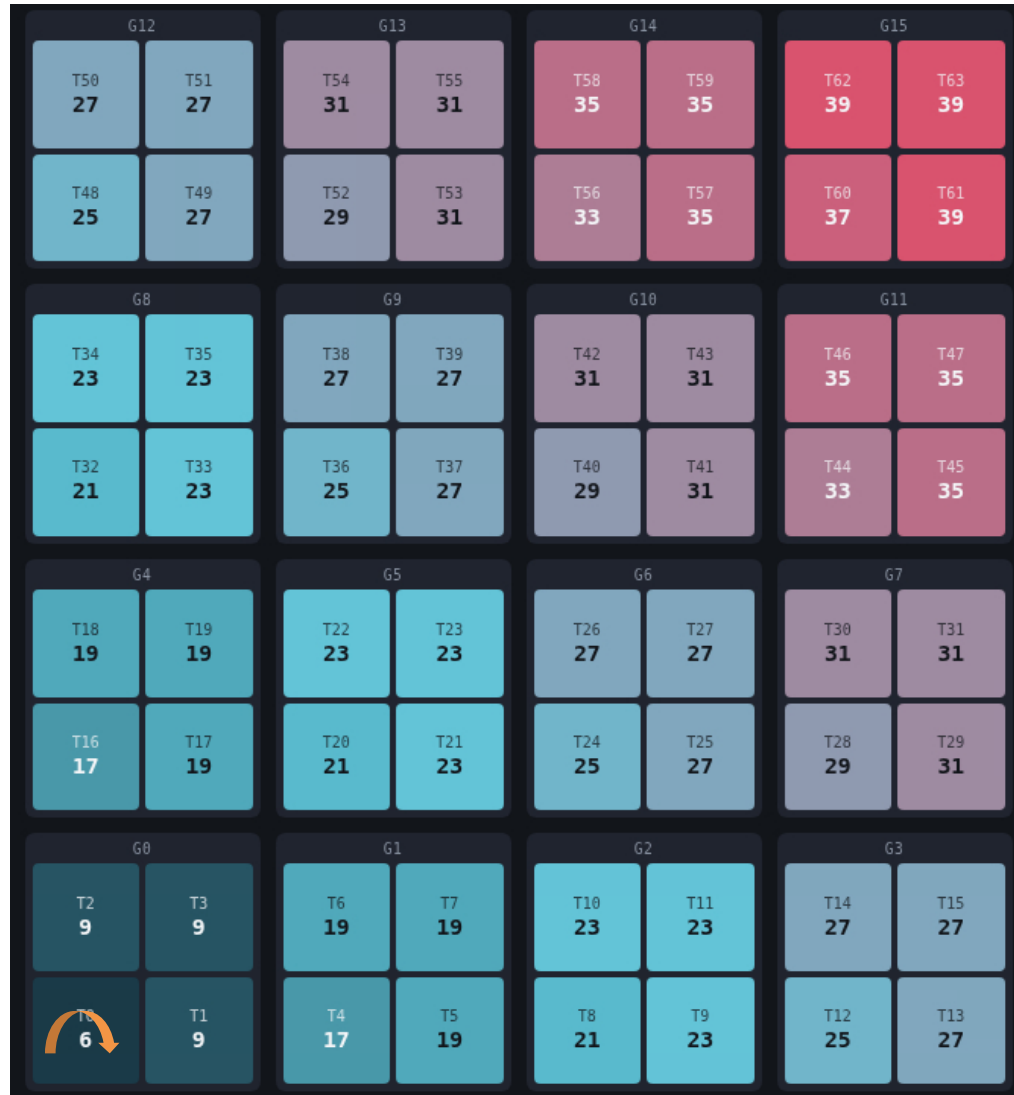


• Latency Path Demonstration

- Each color will represent one cycle
- Separate into request and response path
- Request path starts from core, ends at SRAM
- Response path starts from SRAM, ends back at core
- Each router has 1 cut at input, 1 cut at output



NoC Latency



• Within a tile (6 cyc):

- Request: 4 cyc

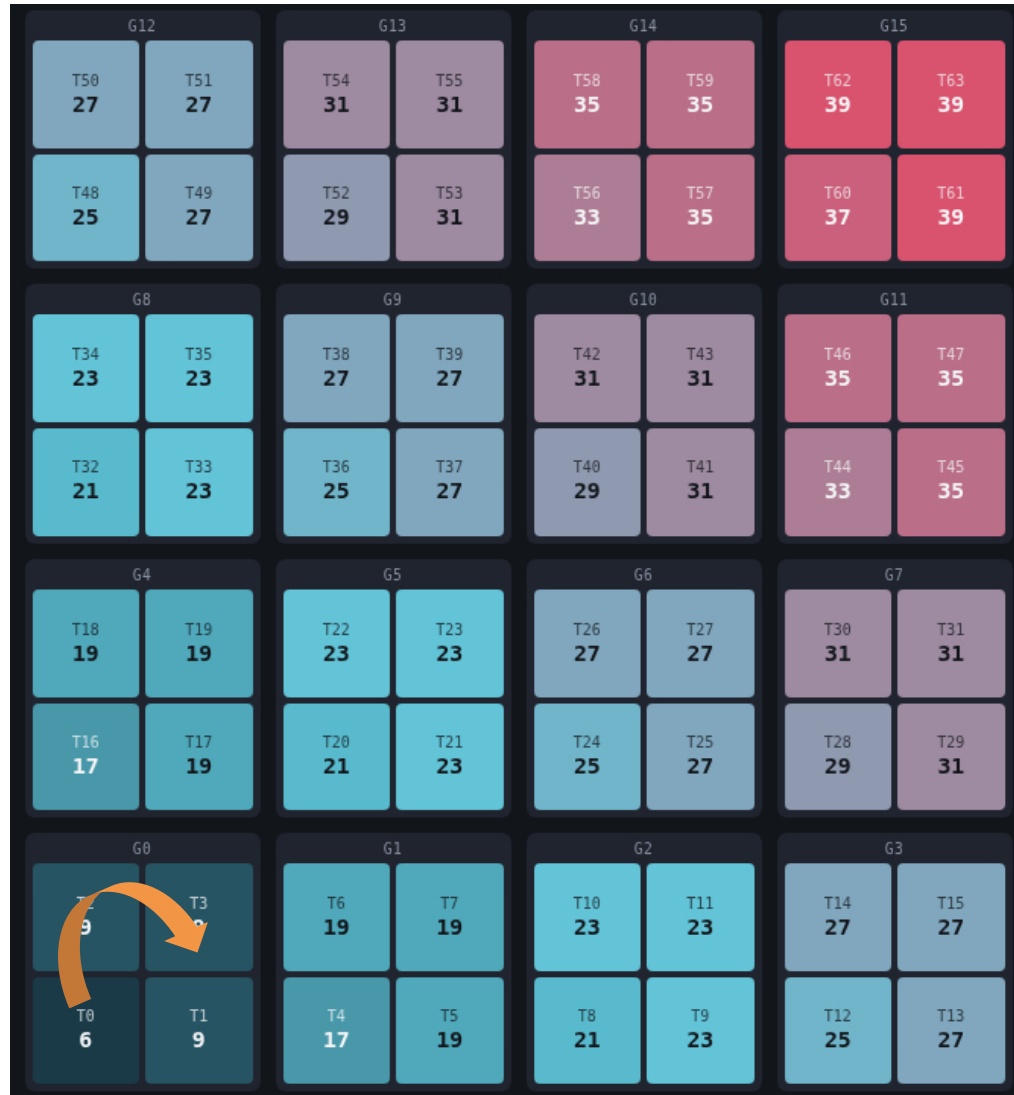
core → cut → scalar xbar (to stack, peripheral or cache) → l1 cache xbar (local or remote) → amo → amo cut → controller → data sram

- Response: 3 cyc (1 overlap with req)

data sram → amo → l1 cache xbar → scalar xbar → cut → core



NoC Latency



• Within a group (9 cyc):

- Request: 5 cyc

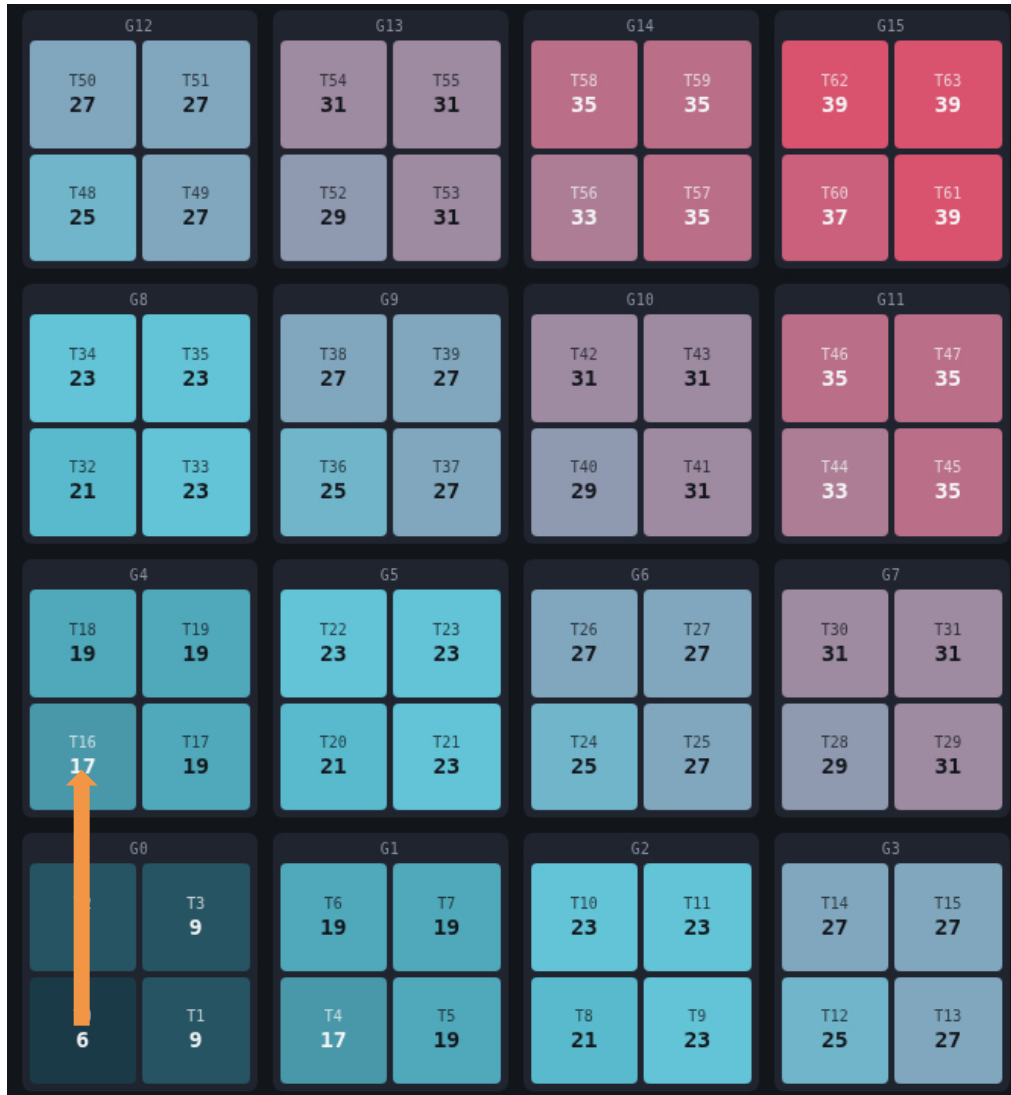
core → cut → scalar xbar (to stack, peripheral or cache) → l1 cache xbar (sender tile) → l1 group xbar → l1 cache xbar (receiver tile) → amo → amo cut → controller → data sram

- Response: 5 cyc (1 overlap with req)

data sram → amo → l1 cache xbar → l1 group xbar → l1 cache xbar → scalar xbar → cut → core



NoC Latency



• Neighbor Group, Same Relative Tile (17 cyc):

- Request: 9 cyc

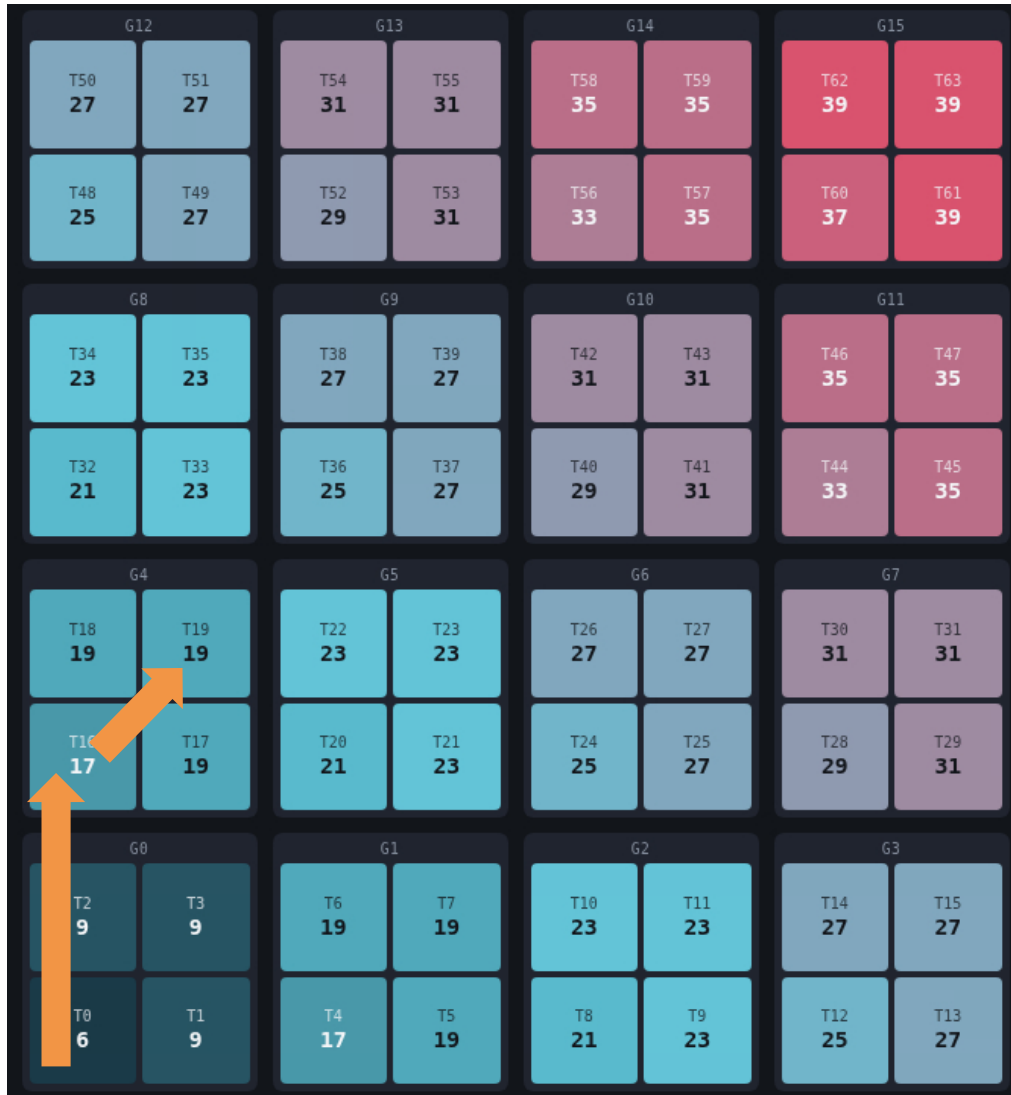
core → cut → scalar xbar (to stack, peripheral or cache) → l1 cache xbar (sender tile) → src router in → src router out → dst router in → dst router out → l1 cache xbar (receiver tile) → amo → amo cut → controller → data sram

- Response: 9 cyc (1 overlap with req)

data sram → amo → l1 cache xbar → l1 group xbar → l1 cache xbar → dst router in → dst router out → src router in → src router out → scalar xbar → cut → core



NoC Latency



• Neighbor Group, Different Relative Tile (19 cyc):

- Request: 10 cyc

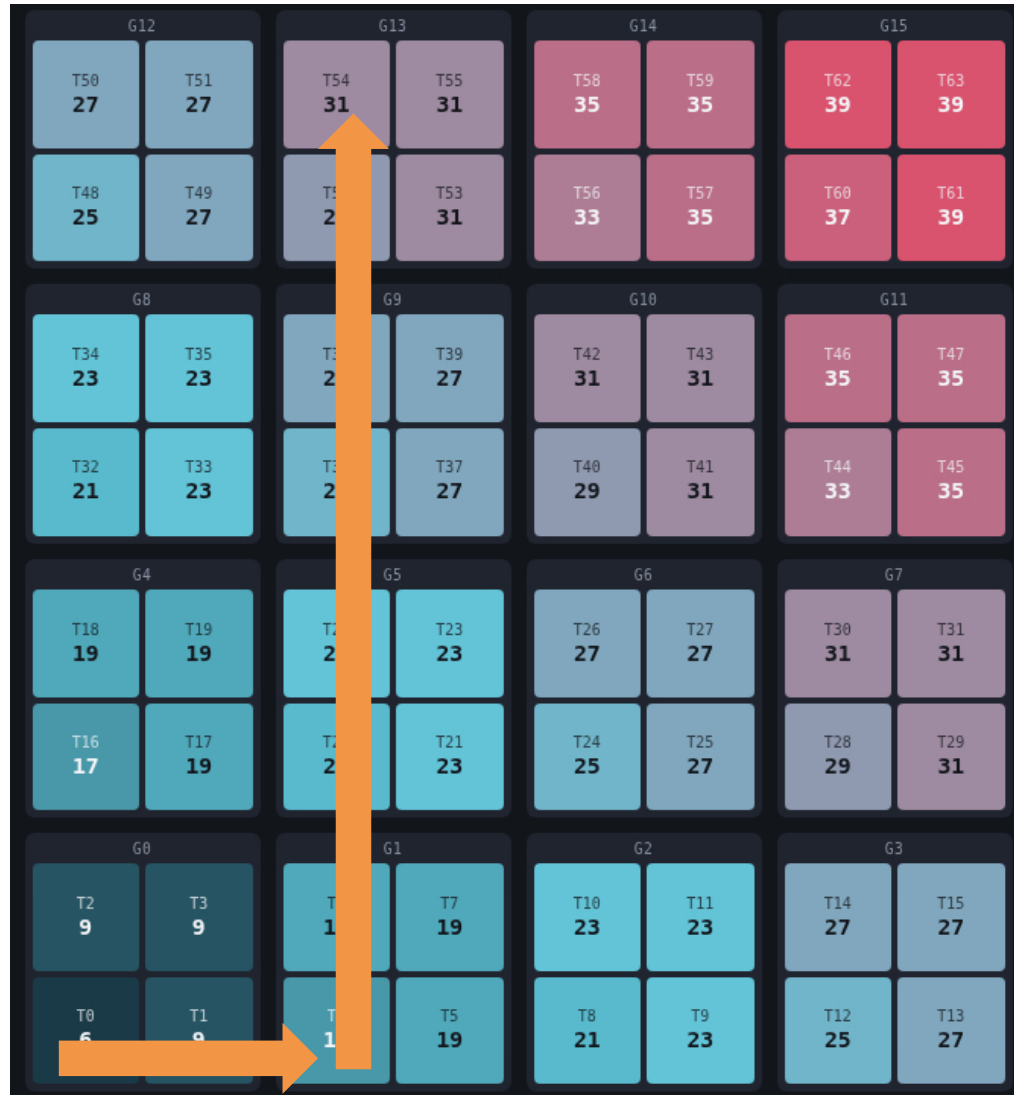
core → cut → scalar xbar (to stack, peripheral or cache) → l1 cache xbar (sender tile) → l1 noc xbar (select which mesh to go) → src router in → src router out → dst router in → dst router out → l1 cache xbar (receiver tile) → amo → amo cut → controller → data sram

- Response: 10 cyc (1 overlap with req)

data sram → amo → l1 cache xbar → l1 group xbar → l1 cache xbar → dst router in → dst router out → src router in → src router out → l1 noc xbar → scalar xbar → cut → core

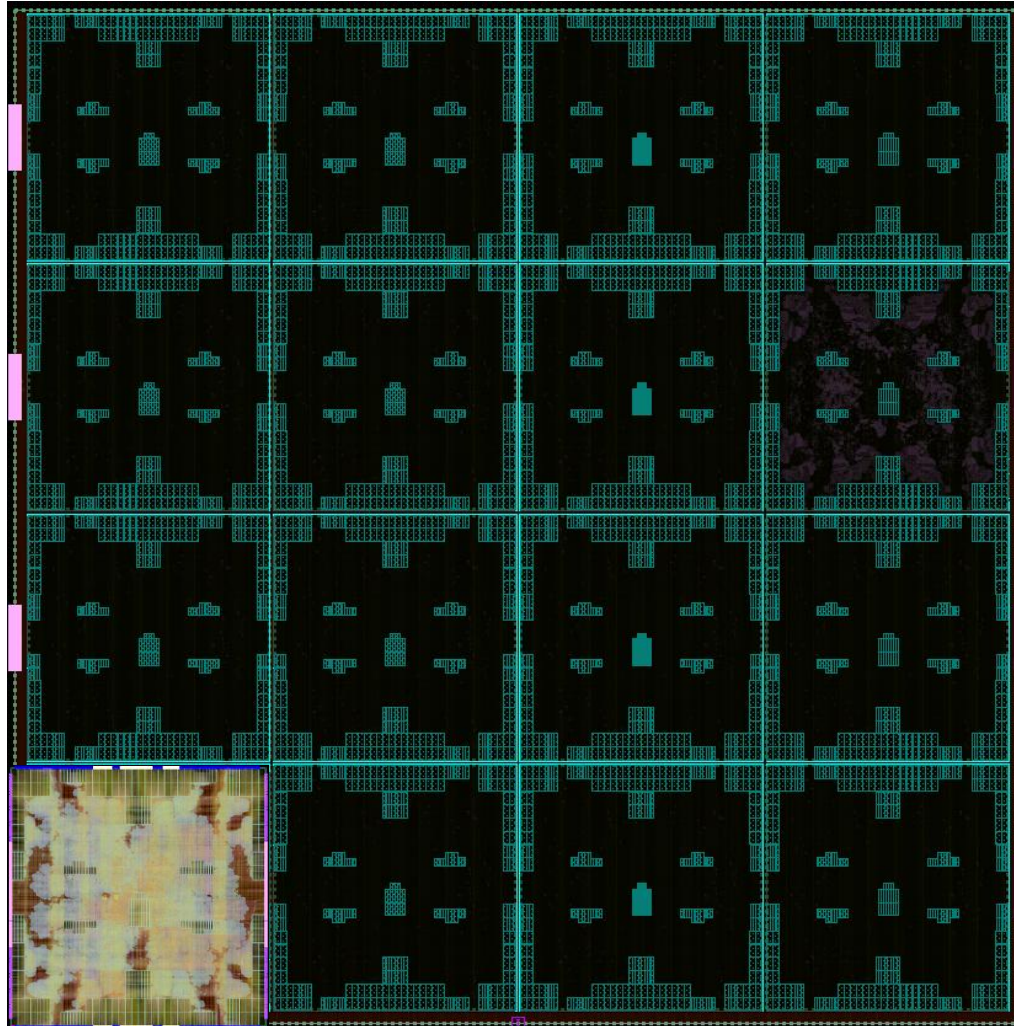


NoC Latency



- Other groups follows the same calculation
 - Just add 2 cycles more on NoC requests, and 2 cycles more on NoC responses





- **Top Level Timing**

- No violation observed between groups
 - Timing on NoC can meet its goal
- Large violation from peripheral to some groups
 - These path are used for configuration registers, will not change during runtime
 - This can be fixed from constraints by using multi-cycle or even false-path
- Also noticed some violations on barrier
 - Adding a cut here is enough to fix the issue

Timing optimization from Insitu cache



- **The current critical path of the whole design is the insitu cache**
 - At 1ns clk target, -0.26ns WNS → 793MHz
- **The critical path**
 - meta-SRAM Q → decoder → Cache_FSM → data-bank we → skew-column grant → meta SRAM address / CEB / ICG-E
- **RTL optimization → More parallel, less logic level**
 - Break serial logic into **parallel logic that derives from registers**, not from other combinational logic
 - **Compare-then-mux** instead of mux-then-compare
 - **Reduce-then-select** instead of select-then-reduce
 - Make the late signal land on **one final gate**, not thread a cascade



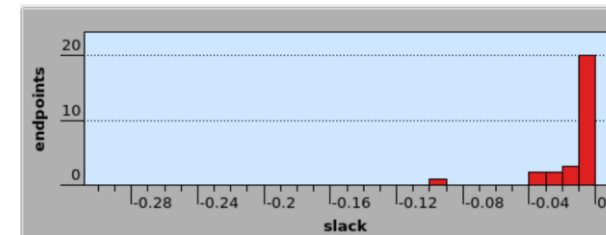
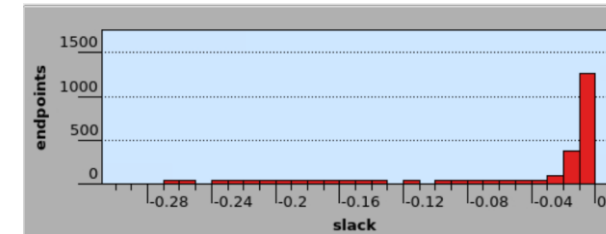
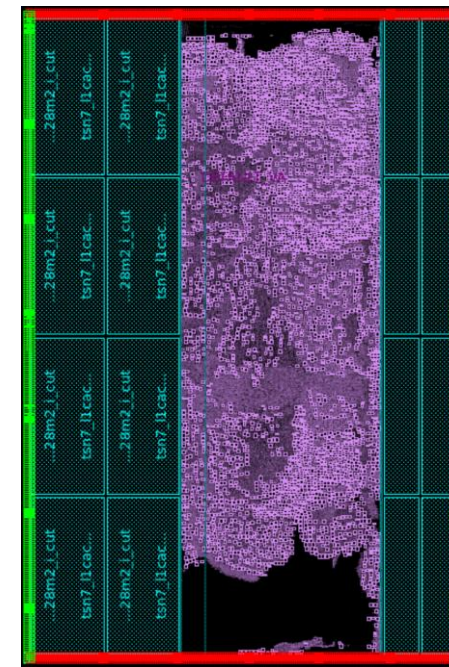
Timing optimization from Insitu cache

- **For fast iteration, out-of-context backend**
 - Only run the synthesis + placement for a single cache bank
 - Simple floorplan
 - Fast iteration: synthesis + placement finish in ~1h

- **Timing improvement**

	Baseline	Optimized	Reduced
WNS	-0.260ns	-0.091ns	-65%
TNS	-31.9ns	-0.27ns	-99%
Violating endpoints	1901	28	-98%
Logic-only area	21974 μm^2	21235 μm^2	-3.36%
ULVT cells	44162	40898	-7.4%

- Scenario: func_ssgnp_0p675v_m40c
- Frequency: 793MHz $\rightarrow$ 917MHz
- Frontend function/performance unchanged



GVSoC Modeling Progress



- **Fix issues on github**

#	Issue	Fix
#42	Strided and indexed vector loads/stores executed as unit stride → silent data corruption	Instructions were never tagged as strided/indexed, so the handlers existed but never ran. Retagged them in the ISA generator
#43	vmv.v declared a false input dependency on v0, serialising unrelated vector work	Added instruction formats that omit the false operand
#38	LR.W returned its value in the wrong register	Forward the correct data pointer before issuing the load
#39	Scale bugs at 64 tiles: a 32-bit shift overflowing, an uninitialised sync flag	64-bit shifts + masks; flag initialised
#37	HTIF status pollers generated timed fabric traffic, saturating a cache bank at high core counts	Made event-driven instead of polling every 1000 cycles

- **Added dual scalar core support in the GVSoC model**

- Multiple Snitch cores in a core complex
- Spatz lock, instruction issue mux
- Locked/unlocked mode



Next Step

- **DATE 2027**

- Previous draft uploaded to github (DATE does not allow arxiv)
- Now working on reshaping the paper for DATE

- **Diyou Side**

- Performance evaluation and debugging
- Paper adjustment and resubmission

- **Zexin Side**

- GVSoC development and calibration under multi-group config
- SW development based on the benchmark plan
- Run the full group backend implementation to check the new timing



HPD vs. InSitu Cache



- **System Configurations**

- Both cache are set to 128KiB, organized in 4 ways
- HPD cache has $16 \times 4 = 64$ -entry MSHR
- TSMC7, 1GHz
- The cluster has maximum $2 \times 4 \times 8B = \mathbf{64B/cyc}$ bandwidth requirement from Spatz
 - Case 1: Connected to L2 SPM (9-cycle latency), **64B/cyc** bandwidth
 - Case 2: Connected to DDR4-2400, **19.2B/cyc** bandwidth



HPD vs. InSitu Cache



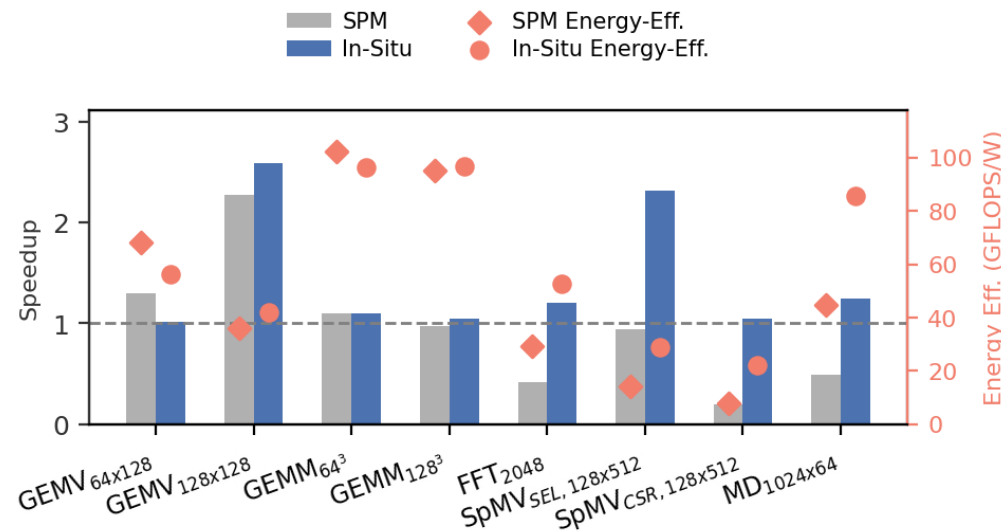
- **Area Comparison**

- InSitu cache controller: 461 KGE
- HPD cache controller: 679 KGE

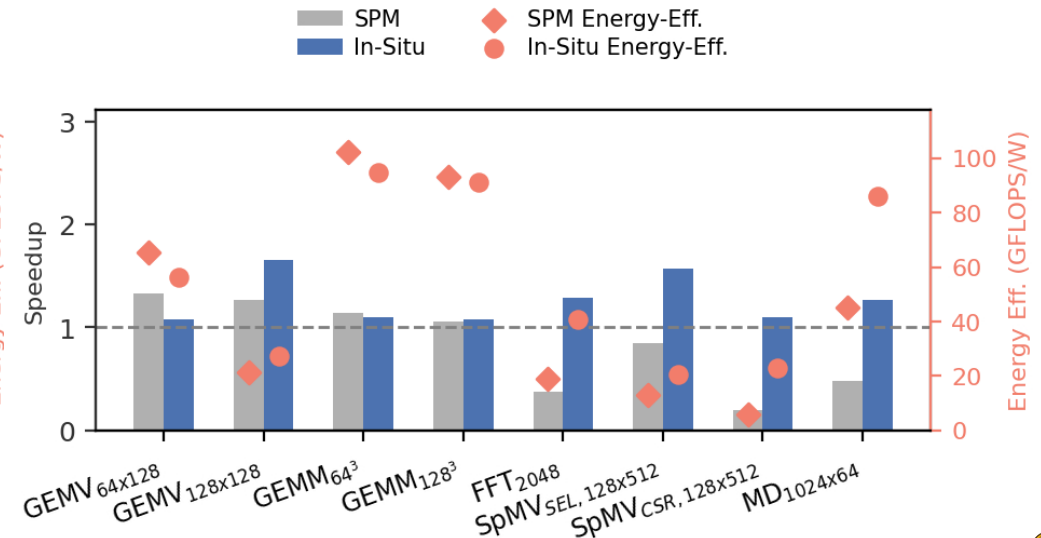
- **Performance Comparison**

- Tested dense kernels, FFT, selective SpMV (used in sparse attention), CSR-SpMV and molecular dynamics
- InSitu beats HPD cache in all test cases

L2 SPM Speedup over HPDCache



DDR4-2400 Speedup over HPDCache



Timeline



		2025					2026							
		11	12	1	2	3	4	5	6	7	8	9	10	11
WP1 ManyCore Architecture	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] Complete													
	Interco. Optimization (multi-group) [RTL] Almost Complete													
	Timing optimization for NoC [RTL] Ongoing by colleague													
	NoC Topology Exploration [RTL, GVSoC] Ongoing by student													
WP2 Cache	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL] Complete													
	Folded Cache [GVSoC + RTL] Complete													



Timeline



	2025								2026						
		11	12	1	2	3	4	5	6	7	8	9	10	11	
WP3 VPE	Shared vector core [RTL + GVSoC] Almost Complete														
WP4 Benchmark	Single-user kernel optimization [SW] Ongoing														
	Multi-user and more scenarios [SW] Ongoing														
Report	Final Report and Benchmark														

L1 Access Latency



- **Problem: increased L1 access latency when scaling up**
- **Solutions**
 - **(Complete, RTL & Physical) Hardware optimization**
Further optimize the interconnection and cache controller to reduce access latency on existing architecture
=> Retiming the current interconnection, remove unnecessary cuts in interco and cache
=> Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
Currently has reduced to 6 cycles:
Core => Xbar => AMO => Ctrl (2) => AMO => Core
 - **(Complete, RTL) Cache partitioning**
Partition the shared L1 to keep data localized to avoid remote access latency
 - **(Complete with some problems, RTL & Physical) WT-L1**
Explore and evaluate a private WT L1 for each core
=> Performance degrade on single tile
=> Needs more work to support larger system



L1 Access Latency



- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
 - **(Ongoing, Physical) Reduce hop latency**
Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
=> Reduce the latency by 1/2
 - **(Planned, Physical) Cut clk-domain (optional)**
If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
=> Effectively cut the latency by 1/3
 - **(Planned, GVSoc & Physical) NoC Topology**
Explore different NoC topologies
=> Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2



PE-Related



- **Problem: Improve scalar core performance**
- **Solutions**
 - **(Complete, RTL & GVSoc) Shared vector core**
Explore two Snitch sharing one Spatz configuration to improve scalar performance
Software complexity needs to be considered, especially on the possible conflicting use of register file.
Discussed Solution: critical section



RLC Workload



- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
 - **(Ongoing, SW) Kernel optimization**
Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
 - **(Planned, SW) Kernel development**
Implement the missing use cases for multi-user



Thank you!

Q&A

